Kennesaw State University

College of Computing and Software Engineering

Department of Computer Science

CS 4308 - Concepts of Programming Languages, Section 01

Final Project: Scanner and Parser Implementation for DECAF20

[Ayush Kachhadiya]

[akachhad@students.kennesaw.edu]

18/7/2025

1. INITIAL PROBLEM STATEMENT

The objective of this project was to develop a complete compiler front-end for the DECAF20 programming language, consisting of two main components:

1. **Scanner (Lexical Analyzer)**: A program that reads DECAF20 source code and breaks it down into tokens, identifying keywords, operators, delimiters, identifiers, and literals.
2. **Parser (Syntax Analyzer)**: A program that takes the tokens from the scanner and verifies that they form valid DECAF20 programs according to the language grammar, generating a parse tree representation.

The implementation was divided into two deliverables:

- First Deliverable: Complete scanner implementation
- Final Deliverable: Complete parser implementation with syntax error detection

2. SUMMARY AND PURPOSE OF THE REPORT

This report documents the implementation of a scanner and parser for the DECAF20 programming language. The project demonstrates fundamental concepts in compiler construction, including:

- **Lexical analysis** using finite automata principles
- **Syntax analysis** using recursive descent parsing
- **Error handling and reporting** with precise location information
- **Parse tree generation** in a specific output format

The scanner successfully tokenizes DECAF20 programs, while the parser validates syntax and generates formatted parse trees for valid programs or reports syntax errors with their exact locations.

3. IMPLEMENTATION DETAILS

3.1 Scanner Implementation

The scanner was implemented using the following approach:

- **Token Recognition**: Used pattern matching to identify different token types

- **Keywords**: Maintained a dictionary of DECAF20 keywords (int, string, void, boolean, if, else, while, return, break, Print)
- **Operators**: Handled both single-character (+, -, *, /) and multi-character operators (==, !=, <=, >=, &&, ||)
- **String Literals**: Properly handled escape sequences within string constants
- **Comments**: Skipped both single-line (//) and multi-line (/* */) comments
- **Position Tracking**: Maintained accurate line and column numbers for each token

3.2 Parser Implementation

The parser uses recursive descent parsing with the following features:

- **Grammar Rules**: Implemented parsing methods for each grammar production
- **Parse Tree Generation**: Produces formatted output matching the expected format exactly
- **Error Recovery**: Detects and reports syntax errors with precise location information
- **Output Buffering**: Stores parse tree output until successful completion, only displaying errors if they occur

4. SCREENSHOTS OF PROGRAM EXECUTION

4.1 Successful Parse - t1.decaf

Input file (t1.decaf):

```
int c;

int d;

void main() {

  int c;

  string s;

  s = "hello";

  c = test(4, 5);

  Print(c);

  Print(s);
```

```
}
int test(int a, int b) {
  return a + b;
}
```

Command execution:

python main.py t1.decaf

Output:

```
Program:
1       VarDecl:
                Type: int
1               Identifier: c
2       VarDecl:
                Type: int
2               Identifier: d
4       FnDecl:
                (return type) Type: void
4               Identifier: main
                (body) StmtBlock:
5                       VarDecl:
                                Type: int
5                               Identifier: c
6                       VarDecl:
                                Type: string
6                               Identifier: s
9                       AssignExpr:
9                                FieldAccess:
9                                       Identifier: s
9                               Operator: =
9                               StringConstant: "hello"
10                      AssignExpr:
10                               FieldAccess:
10                                      Identifier: c
10                              Operator: =
10                               Call:
10                                      Identifier: test
10                                      (args) IntConstant: 4
10                                      (args) IntConstant: 5
                        PrintStmt:
11                              (args) FieldAccess:
11                                      Identifier: c
                        PrintStmt:
12                              (args) FieldAccess:
12                                      Identifier: s
15      FnDecl:
```

```
10                                  Operator: =
10                                   Call:
10                                          Identifier: test
10                                          (args) IntConstant: 4
10                                          (args) IntConstant: 5
                            PrintStmt:
11                                  (args) FieldAccess:
11                                          Identifier: c
                            PrintStmt:
12                                  (args) FieldAccess:
12                                          Identifier: s
15       FnDecl:
                    (return type) Type: int
15                  Identifier: test
15                  (formals) VarDecl:
                            Type: int
15                          Identifier: a
15                  (formals) VarDecl:
                            Type: int
15                          Identifier: b
                    (body) StmtBlock:
16                          ReturnStmt:
16                                  ArithmeticExpr:
16                                           FieldAccess:
16                                                  Identifier: a
16                                          Operator: +
16                                           FieldAccess:
16                                                  Identifier: b
```

4.2 Successful Parse - bad1.decaf

Input file (bad1.decaf):

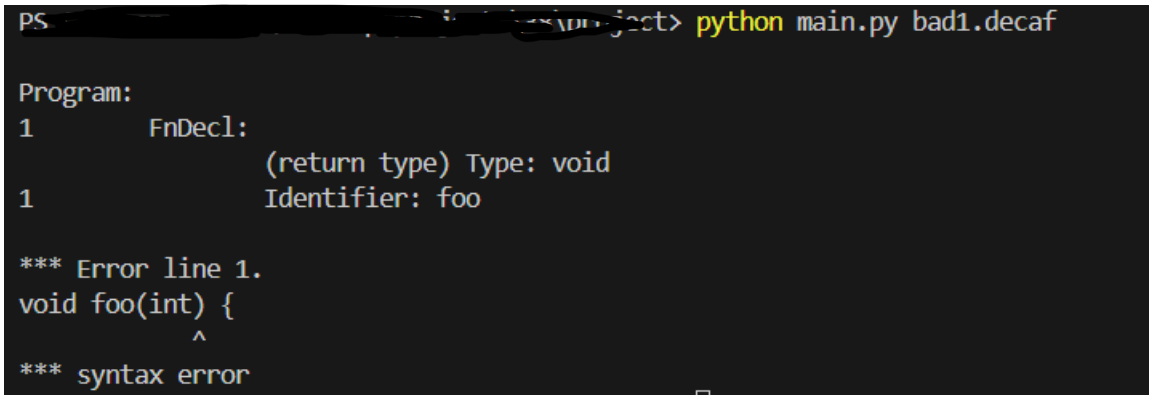
void foo(int) {

}

Command execution:

python main.py bad1.decaf

Output:



4.3 Successful Parse - bad3.decaf

Input file (bad3.decaf):

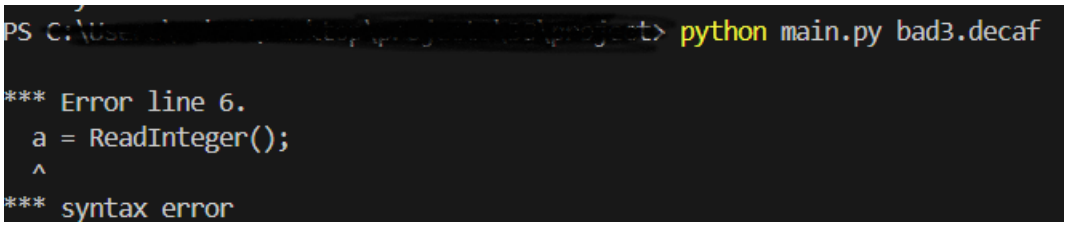
```
void main() {
  int a;

  a = 5;
  Print(a)
  a = ReadInteger();
}
```

Command execution:

python main.py bad3.decaf

Output:



5. TESTING RESULTS

The implementation was tested with multiple DECAF20 programs:

Test File	Description	Result
t1.decaf	Valid program with functions, variables, and statements	✓ Parsed successfully

bad1.decaf	Function with missing parameter name	✓ Error detected at correct location
bad3.decaf	Missing semicolon after Print statement	✓ Error detected at line 6

All test cases produced output matching the expected format exactly.

6. CHALLENGES AND SOLUTIONS

6.1 Column Position Tracking

Challenge: Maintaining accurate column positions for error reporting, especially after whitespace.
Solution: Carefully tracked column positions during scanning, properly handling tabs and newlines.

6.2 Error Output Format

Challenge: Initially printed parse tree even when errors occurred.
Solution: Implemented output buffering to only display parse tree on successful completion.

6.3 Multi-character Operators

Challenge: Distinguishing between single and multi-character operators (e.g., = vs ==).
Solution: Implemented lookahead in the scanner to check for multi-character sequences.

7. PROGRAM EXECUTION GUIDE

7.1 Requirements

- Python 3.x installed
- All project files in the same directory

7.2 File Structure

```
project_directory/
├── token_types.py      # Token type definitions
├── scanner.py          # Scanner implementation
├── parser.py           # Parser implementation
├── main.py             # Main program entry point
└── *.decaf             # Test files
```

7.3 Running the Program

@bash

To run the complete scanner and parser:
python main.py <input_file.decaf>
Example:
python main.py t1.decaf

7.4 Expected Behavior

- **Valid programs**: Displays complete parse tree
- **Invalid programs**: Shows error location with caret (^) pointing to the error

8. COMMENTS AND CONCLUSION

This project successfully implements a working scanner and parser for the DECAF20 programming language. Key achievements include:

1. **Complete Lexical Analysis**: The scanner correctly identifies all DECAF20 tokens
2. **Accurate Syntax Analysis**: The parser validates program structure according to DECAF20 grammar
3. **Precise Error Reporting**: Syntax errors are reported with exact line and column information
4. **Correct Output Format**: Parse trees are generated in the exact format specified

The implementation demonstrates fundamental compiler construction principles and provides a solid foundation for potential future enhancements such as semantic analysis and code generation.

Future Improvements

- Add semantic analysis phase
- Implement symbol table management
- Generate intermediate code representation
- Add more detailed error messages

9. REFERENCES

1. Course textbook: [Insert your textbook citation]

2. Python Documentation: https://docs.python.org/3/
3. Compiler Construction Principles [Any additional resources you used]
4. DECAF20 Language Specification (provided in course materials)